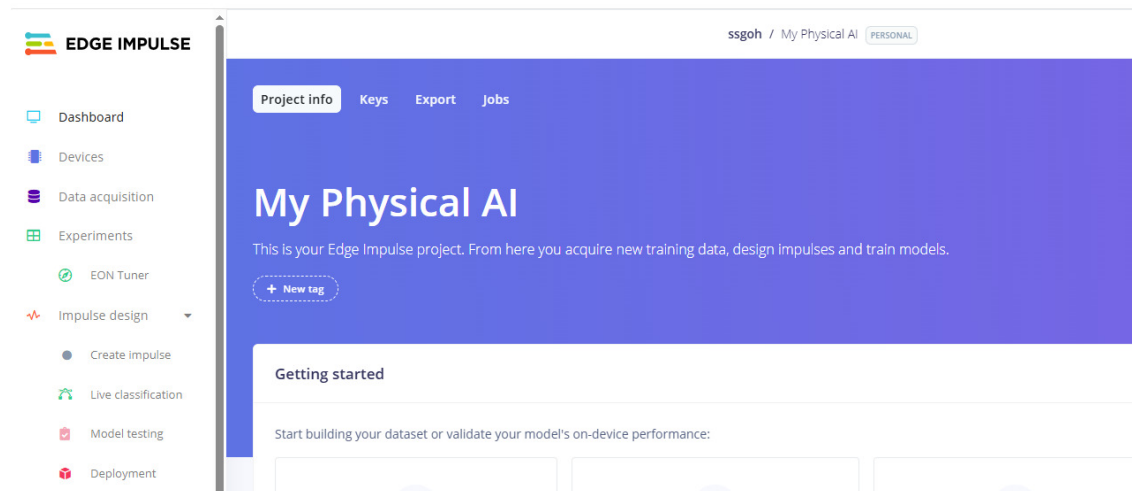


<https://www.edgeimpulse.com>

Click Login and Login with Google Account

Click Create New Project

Enter Project Name [My Physical AI] and click Create New Project

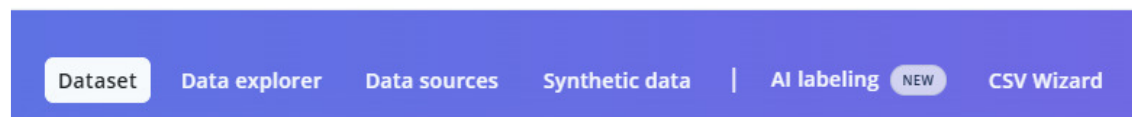


My Physical AI

This is your Edge Impulse project. From here you acquire new training data, design impulses and train models.

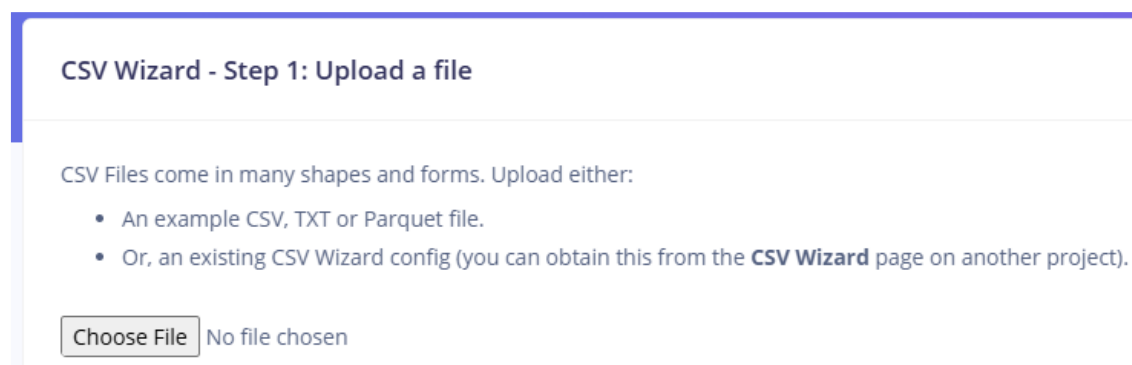
Click Data Acquisition on the left panel

You will see this



Click CSV Wizard

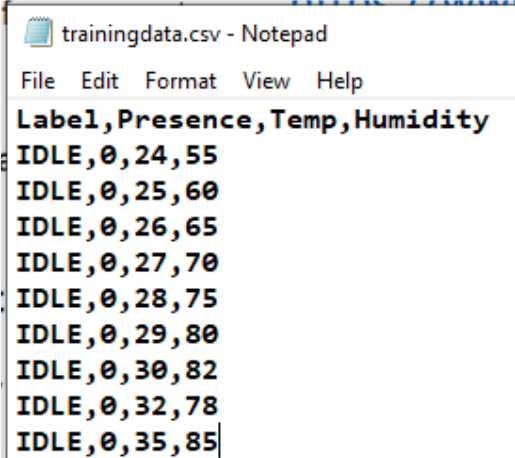
You will see this



Click Choose File and upload the training data that you have prepared.

In this example it is **trainingdata.csv** that is in C:\PhysicalAI folder

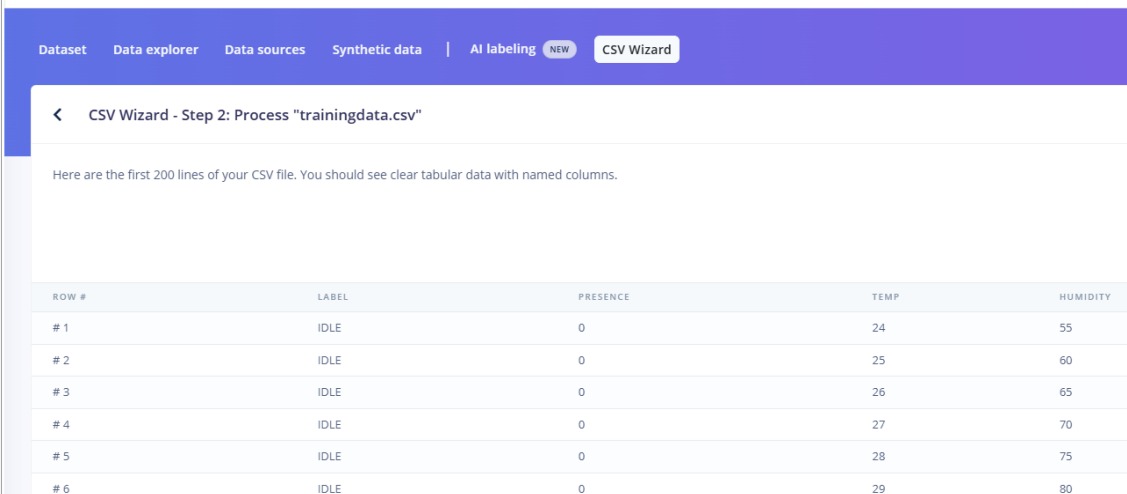
This is a sample of the data inside



```
trainingdata.csv - Notepad
File Edit Format View Help
Label,Presence,Temp,Humidity
IDLE,0,24,55
IDLE,0,25,60
IDLE,0,26,65
IDLE,0,27,70
IDLE,0,28,75
IDLE,0,29,80
IDLE,0,30,82
IDLE,0,32,78
IDLE,0,35,85
```

Once the file is chosen, click Upload File

You will see your data uploaded on the screen



ROW #	LABEL	PRESENCE	TEMP	HUMIDITY
# 1	IDLE	0	24	55
# 2	IDLE	0	25	60
# 3	IDLE	0	26	65
# 4	IDLE	0	27	70
# 5	IDLE	0	28	75
# 6	IDLE	0	29	80

Scroll all the way down and look for this button and click on it.

Looks good, next

You will be asked if this is time series data. Check the No option and click the Great, let's look at your values

< CSV Wizard - Step 3: About your data

Let's learn a bit about the structure of your data.

Is this time-series data?

- ☐ Yes, this is time-series data (either raw sensor data, or processed features)
- ☒ No

[Different format?](#)

Great, let's look at your values

You will see this : Make sure the Label Column is checked and all data used for training is checked as well. See Example below.

< CSV Wizard - Step 4: About your values

Almost there. Let's parse your values.

Do you have a column that contains the label (the value you want to predict)?

- ☒ Yes, it's
- ☐ No (no worries, you can provide this when you upload data)

Which columns contain your values?

- ☒ Presence
- ☒ Temp
- ☒ Humidity

[Different format?](#)

Finish wizard

Then Click Finish Wizard

You will see this confirmation screen



Any CSV files that you upload into your project - whether it's through the uploader, the CLI, the API or through data sources - will now be processed according to the rules you set up here.

 Upload some data

This is the confusion part. After uploading the data using the csv wizard, I am asked to upload some data (again??). This is the explanation from Gemini

I understand the confusion! It feels like you've just finished a task, only to be asked to do it again.

The short answer is: **You haven't actually uploaded your data files yet.**

Using our **Chef analogy**, you just finished **building the recipe book** (the CSV Wizard), but you haven't brought the **actual groceries** (your data files) into the kitchen.

What Happened

The **CSV Wizard** is a setup tool, not the uploader itself. Because CSV files can be messy and organized in many different ways, Edge Impulse needs you to "teach" it how to read your specific files first.

1. **The Wizard (What you just did):** You told Edge Impulse which column is "Temperature," which is "PIR," and which is the "Label." You essentially created a **template**.
2. **The Upload (The next step):** Now that Edge Impulse has the template, it is ready for you to dump your actual `.csv` files into the system so it can process them using the rules you just set up.

What to do now

You don't have to do the Wizard again. You are on the right track!

1. Click that blue "**Upload some data**" button.
2. Select the actual CSV files from your computer that contain your PIR, Temperature, and Humidity readings.
3. Edge Impulse will now use the "rules" you just created to automatically sort that data into your training set.

How to check if it worked

Once you upload those files, look at the **left-hand sidebar** in your image:

- Click on **Data Acquisition**.
- You should see a list of your files appearing there.
- If you see your data rows with labels like "Fan_On" or "Fan_Off," your "Chef" has officially received his ingredients!

Upload the **trainingdata.csv** that is in C:\PhysicalAI folder AGAIN

Upload data

You can upload CBOR, JSON, CSV, Parquet, WAV, JPG, PNG, AVI or bounding boxes, labels, and/or metadata. View [Uploader docs](#) to formats, such as pandas or numpy; or [import data directly from a](#)

CSV and Parquet files will be processed using the active CSV Wizard

Upload mode

☒ Select individual files [?](#)

☐ Select a folder [?](#)

Select files

Choose Files

trainingdata.csv

Upload into category

☒ Automatically split between training and testing [?](#)

☐ Training

☐ Testing

Label

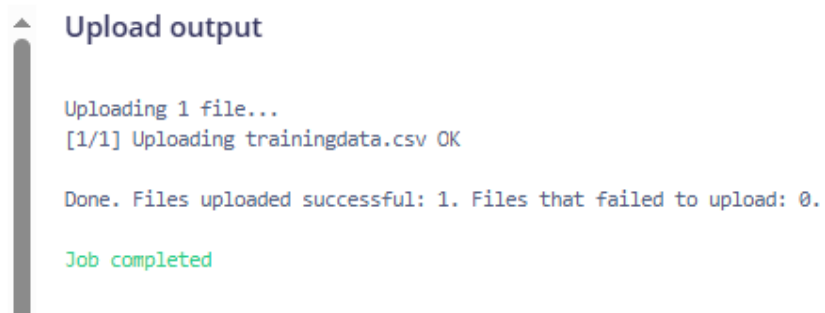
i CSV and Parquet files will be labeled using the 'Label' column, as configured in the CSV wizard. [Update configuration](#) [?](#)

Leave the Automatically split between training and testing selection alone. Don't change

Upload data

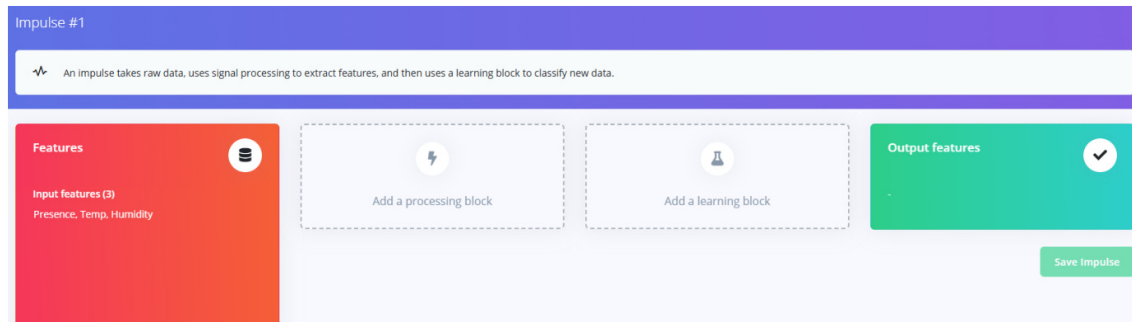
Click the button to upload

If everything goes on well you will see this : Job Completed



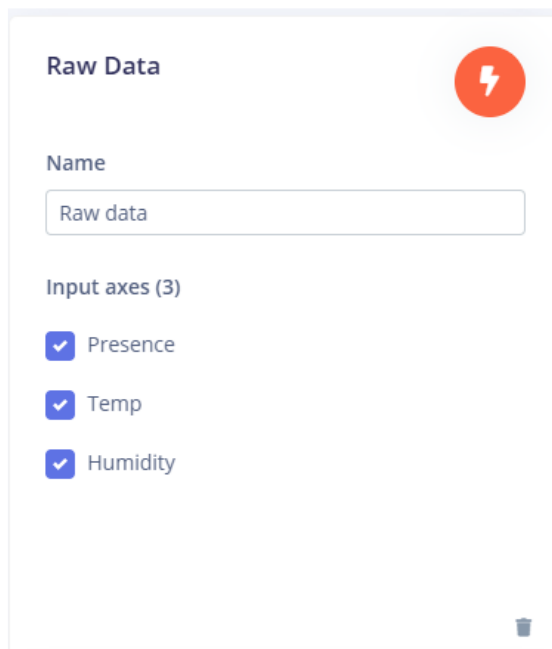
Click [x] to close the pop up

Go to the left pane and click Create Impulse. You will see this screen



Click Add a Processing Block

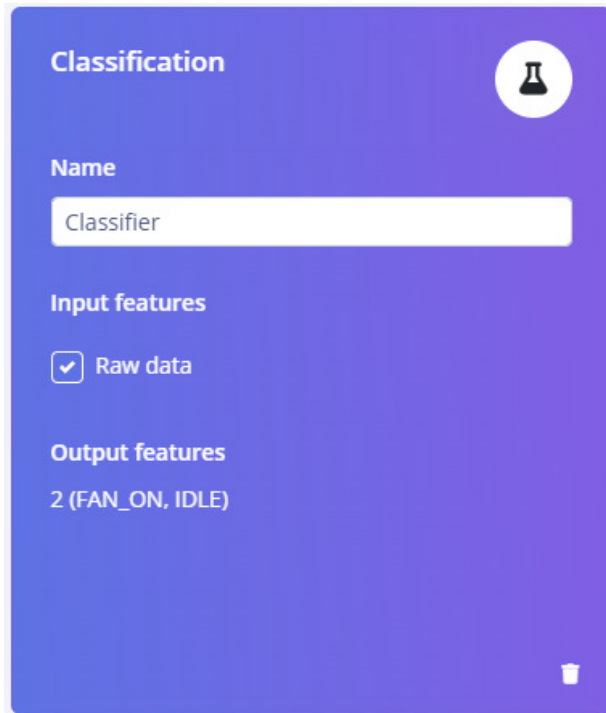
Choose Raw Data and Click ADD and you will see this updated on the processing block.



Next Click Add a Learning Block

Select Classification and click Add.

You will see the Learning Block updated



The image shows a configuration panel for a 'Classification' learning block. The panel has a purple header with the title 'Classification' and a flask icon. Below the header, there is a 'Name' field with the text 'Classifier'. Under 'Input features', the 'Raw data' checkbox is checked. Under 'Output features', it shows '2 (FAN_ON, IDLE)'. A trash icon is located at the bottom right of the panel.

Once done, save this impulse by clicking on

Save Impulse button

You will see this

✓ Successfully stored impulse. Configure the signal processing and learning blocks in the navigation bar. ✕

Click Raw Data to view the training data

Parameters

Generate features

Raw data

KEY	VALUE
Presence	1
Temp	35
Humidity	87

Raw features

1, 35, 87

Parameters

Scaling

Scale axes1

Save parameters

DSP result

Processed features

1, 35, 87

On-device performance

PROCESSING TIME

1 ms.

Save parameters

Click Save parameters

You will be brought to this screen

Parameters

Generate features

Training set

Data in training set

1m 0s

Classes

2 (FAN_ON, IDLE)

Calculate feature importance

☐

Normalize features ?

Don't normalize data

Generate features

Feature generation output

(0)

Click the **Generate features** button. This turns your text rows into a "map" the AI can read.

Give some time for the generation. At the end of it you will see this screen

Feature generation output

```
Creating windows from files...
[ 1/60] Creating windows from files...
[ 7/60] Creating windows from files...
[60/60] Creating windows from files...
Creating windows from files OK

Created 60 windows: FAN_ON: 32, IDLE: 28

Creating features...
[ 1/60] Creating features...
[60/60] Creating features...
Created features

Job completed (success)
```

Next click Classifier on the left pane.

You will see this screen

Neural Network settings

Training settings

Number of training cycles ?

30

Use learned optimizer ?

☐

Learning rate ?

0.0005

Training processor ?

CPU

Advanced training settings

Neural network architecture

Neural network

Complete architecture / classic ML

Input layer (3 features)

Dense layer (20 neurons)

Dense layer (10 neurons)

Add an extra layer

Output layer (2 classes)

Save & train

Click Save and train. Give some time for the training. At the end you will see this screen

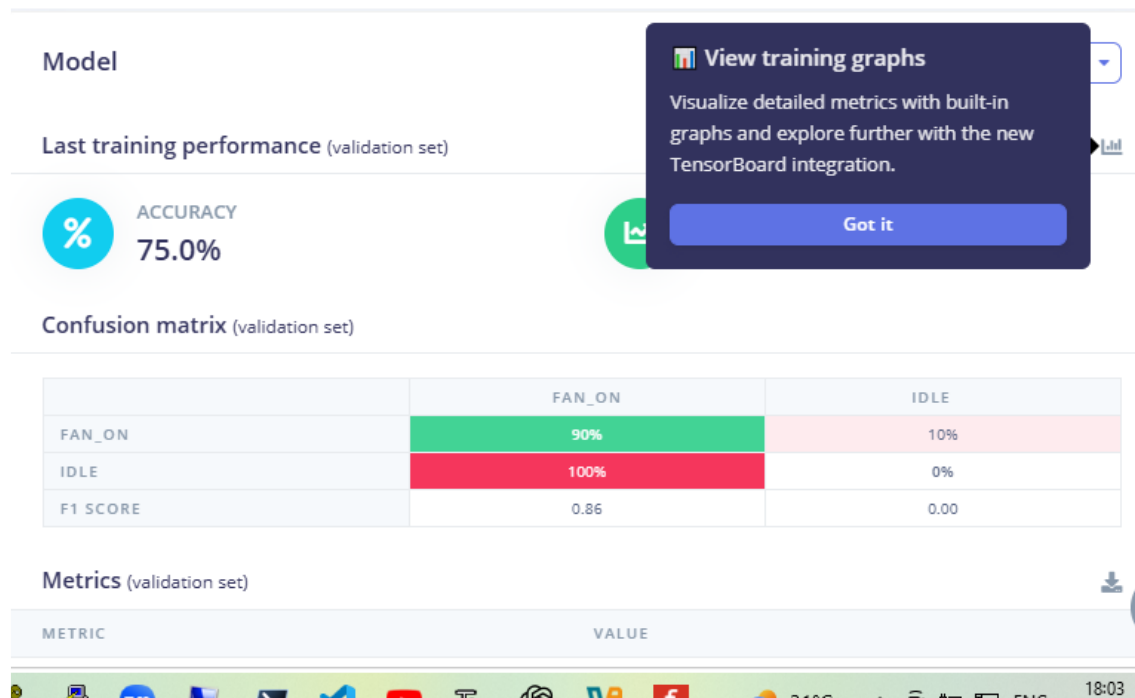
Training output

```
Calculating performance metrics...
Calculating inferencing time...
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Calculating inferencing time OK
Calculating float32 accuracy...
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Calculating int8 accuracy...
Extracting TensorBoard logs...
Extracting TensorBoard logs OK
```

Model training complete

Job completed (success)

And the following training performance



END OF TRAINING

Once the training is done and you are happy with the accuracy:

1. Go to **Deployment**.
2. Select **Raspberry Pi 4**.
3. Click **Build**.
4. It will give you the `model.eim` file you need for your Python script!

When you click Deployment, you will see this screen

Configure your deployment

You can deploy your impulse to any device. This makes the model run without an internet connection, minimizes latency, and runs with minimal power consumption. [Read more.](#)

Deployment target



C++ library

A portable C++ library with no external dependencies, which can be compiled with any modern C++ compiler.

Inference engine



EON™ Compiler

Same accuracy, 55% less RAM, 66% less ROM.

Model optimizations and performance

Change the Deployment target to the Linux version used by Raspberry Pi 4 as I am using a RPi 4 as my edge server.



brainchip

Linux (AARCH64 with Brainchip AKD1000 or AKD1500)

An EIM binary for Linux (AARCH64) target with BrainChip AKD1000 or AKD1500 devices.



Linux (AARCH64 with Qualcomm QNN)

An EIM binary for Linux (AARCH64) with Qualcomm NN accelerators, that implements the Edge Impulse Linux protocol. For example: Dragonwing RB3 Gen 2 or Rubik Pi 3



Linux (AARCH64)

An EIM binary for Linux (AARCH64) that implements the Edge Impulse Linux protocol.



Linux (ARMv7)

An EIM binary for Linux (ARMv7) that implements the Edge Impulse Linux protocol.

brainchip

Linux (x86_64 with Brainchip AKD1000 or AKD1500)

An EIM binary for Linux (x86_64) target with BrainChip AKD1000 or AKD1500 devices.

The recommended selection for RPi4 is Linux(AARCH64).

Click Build after selecting this option

Build

Give some time for the Build process. At the end you will see the build completion pop up

```
posix/ei_classifier_porting.o edge-impulse-sdk/porting/posix/debug_log.o edge-impulse-sdk/porting/mingw32/ei_classifier_porting.o edge-impulse-sdk/porting/mingw32/debug_log.o third_party/base64/at_base64_lib.o third_party/jpeg/JPEGENC.o edge-impulse-sdk/tensorflow/lite/kernels/custom/tree_ensemble_classifier.o -o build/linux-impulse-runner -lm -lstdc++ -L./tflite/linux-aarch64 -Wl,--no-as-needed -ldl -ltensorflow-lite -lfarmhash -lfft2d_fftsig -lfft2d_fftsig2d -lruby -lXNNPACK -lcuinfo -lpthreadpool -lpthread -lrt -lcurl
Building binary OK

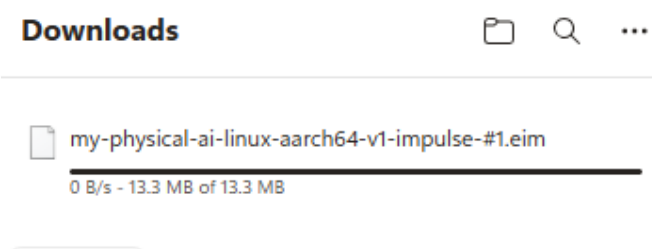
Backing up deployment...

Backing up deployment OK

Download URL: https://studio.edgeimpulse.com/v1/api/970704/deployment/history/1/download

Job completed (success)
```

You will also see that the model is being downloaded to your downloads folder



And finally, a prompt telling you that all is done correctly



Built Linux (AARCH64) model

 [Learn how to use this model](#)

You should see a file with an .eim extension like this in the downloads folder.

To simply it you can rename it models.eim when deploying it in the code running the edge server

my-physical-ai-linux-aarch64-v1-impulse-#1.eim

rename my-physical-ai-linux-aarch64-v1-impulse-#1.eim in Downloads folder to model.eim

VNC Transfer it to raspberry pi edge server. The ip address is 192.168.10.119. user name of the Raspberry Pi is raspberry and the password is pi.

Use this command line to transfer from PC to raspberry pi edge server

```
C:\Users\iamss\Downloads>scp model.eim raspberry@192.168.10.119:/home/raspberry/
```

This is a sample of the edge server code. edgeserver.py

```
from flask import Flask, request, jsonify
from edge_impulse_linux.runner import ImpulseRunner
import os

app = Flask(__name__)

# Use the absolute path we verified yesterday
model_file = '/home/raspberry/model.eim'

# Initialize the runner
runner = ImpulseRunner(model_file)
runner.init()

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.json
        features = data['features']

        # Run the AI model
        res = runner.classify(features)

        # 'result' contains the scores for IDLE and FAN_ON
        predictions = res['result']['classification']

        # Pick the label with the highest score
        label = max(predictions, key=predictions.get)
        confidence = predictions[label]

        print(f"Input: {features} -> Prediction: {label} ({confidence*100:.2f}%)")

        return jsonify({
            "label": label,
            "confidence": predictions # Returns scores for both classes
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 400

if __name__ == '__main__':
```

```
# Listen on all network interfaces on port 5000
app.run(host='0.0.0.0', port=5000)
```